

Αρχές Γλωσσών Προγραμματισμού & Μεταφραστών

Τμήμα Μηχανικών Η/Υ & Πληροφορικής — Πανεπιστήμιο Πατρών

Εαρινό Εξάμηνο 2026

Εργαστηριακή Άσκηση — Λεξικός & Συντακτικός Αναλυτής `mySQLq` (Flex / Bison)

Στοιχεία Ομάδας

Ονοματεπώνυμο	ΑΜ	Έτος	e-mail
Κάνδηλας Γεώργιος	1115510	2ο	up1115510@upatras.gr
Καρπέτας Απόστολος	1115507	2ο	up1115507@upatras.gr

1. Εισαγωγή

Η εργασία υλοποιεί έναν πλήρη λεξικό και συντακτικό αναλυτή για την ψευδογλώσσα **mySQLq** χρησιμοποιώντας τα εργαλεία **Flex** (lexer) και **Bison** (parser). Η υλοποίηση χωρίζεται σε τρία ερωτήματα που το καθένα επεκτείνει το προηγούμενο:

Ερώτημα	Βαρύτητα	Τι υλοποιείται
Q1	60%	Λεξική + συντακτική ανάλυση· έλεγχος γραμματικής δομής, διαγνωστικά λάθους με αριθμό γραμμής
Q2	15%	Σημασιολογική ανάλυση· symbol table, έλεγχος ύπαρξης πινάκων/στηλών, type checking
Q3	25%	Επέκταση με JOIN, AS aliases, qualified αναφορές (table.column), ambiguity check

Ροή Εκτέλεσης

Το πρόγραμμα καλείται ως `myParser.exe file_name`. Σε πρώτο πέρασμα φορτώνει όλες τις γραμμές του αρχείου σε buffer (για εκτύπωση σφαλμάτων). Στη συνέχεια ο lexer τροφοδοτεί tokens στον parser· εφόσον επιτύχει η ανάλυση, εκτυπώνεται το αρχείο και το μήνυμα «*Syntactic analysis successful.*». Σε περίπτωση λάθους, εκτυπώνονται οι γραμμές μέχρι το σημείο σφάλματος και διαγνωστικό μήνυμα με τον αριθμό γραμμής.

2. Γραμματική BNF

Σύμβολα: **κεφαλαία** = τερματικά (tokens), **πεζά** = μη-τερματικά, ϵ = κενή παραγωγή.

2.1 Βασική Γραμματική (Q1 & Q2)

```
stmt_list      →  $\epsilon$  | stmt_list stmt

stmt           → create_stmt | select_stmt

create_stmt    → CREATE TABLE IDENT ( col_def_list ) ;

col_def_list   → col_def | col_def_list , col_def

col_def        → IDENT INT
                | IDENT FLOAT
                | IDENT VARCHAR ( INT_LIT )

col_expr_list  → * | col_ref_list

col_ref_list   → col_ref | col_ref_list , col_ref

opt_where      →  $\epsilon$  | WHERE condition

condition      → condition OR condition
                | condition AND condition
                | NOT condition
                | ( condition )
                | col_ref comp_op literal
                | col_ref IN ( in_literal_list )
                | col_ref NOT IN ( in_literal_list )

comp_op        → = | != | > | < | >= | <=

literal        → INT_LIT | FLOAT_LIT | STR_LIT

in_literal_list → literal | in_literal_list , literal

opt_group      →  $\epsilon$  | GROUP BY col_ref_list
```

```
opt_order      → ε | ORDER BY col_ref_list
opt_limit      → ε | LIMIT INT_LIT
```

2.2 Γραμματική Q1

```
select_stmt → SELECT col_expr_list FROM IDENT
            opt_where opt_group opt_order opt_limit ;

col_ref     → IDENT | IDENT . IDENT
```

2.3 Γραμματική Q2

```
select_stmt      → SELECT select_col_ref_list FROM IDENT
                  opt_where opt_group opt_order opt_limit ;
                  | SELECT * FROM IDENT
                  opt_where opt_group opt_order opt_limit ;

select_col_ref_list → col_ref | select_col_ref_list , col_ref

col_ref           → IDENT          -- χωρίς qualified ref (δεν υπάρχουν JOIN/alias)
```

2.4 Γραμματική Q3 (επεκτάσεις)

```
select_stmt      → SELECT col_expr_list FROM table_ref join_list
                  opt_where opt_group opt_order opt_limit ;

table_ref        → IDENT | IDENT AS IDENT

join_list        → ε
                  | join_list JOIN table_ref ON qualified_col_ref = qualified_col_ref

qualified_col_ref → IDENT . IDENT

col_ref          → IDENT | IDENT . IDENT
```

3. Λεξικός Αναλυτής (Flex)

Το αρχείο `lexer.l` αναγνωρίζει τα παρακάτω tokens. Τα keywords είναι **case-insensitive** (μέσω κατάλληλου regex ή μετατροπής σε lowercase). Τα identifiers είναι **case-sensitive**.

Token	Pattern / Περιγραφή
IDENT	<code>[a-zA-Z][a-zA-Z0-9_]*</code> (αν δεν ταιριάζει keyword)
INT_LIT	<code>-?[0-9]+</code>
FLOAT_LIT	<code>-?[0-9]+\.[0-9]+</code>
STR_LIT	<code>'[^']*'</code> (αφαιρούνται τα single quotes στην τιμή)
Keywords	SELECT FROM WHERE GROUP ORDER BY LIMIT CREATE TABLE JOIN ON AS IN AND OR NOT INT FLOAT VARCHAR
Operators	<code>= != > < >= <=</code>
Punctuation	<code>, . ; () *</code>
Σχόλια	<code>--[^\n]*</code> και <code>/*...*/</code> — αγνοούνται (skip)
Whitespace	Αγνοείται (skip)

Τα block comments χειρίζονται με regex `"/*"([^\*]|\\*[^\*\/])*\*+/"` ώστε να αποφεύγονται false matches σε multiline περιπτώσεις.

4. Ερώτημα 1 — Συντακτική Ανάλυση

Ο parser του Q1 ελέγχει **μόνο τη συντακτική ορθότητα** — δεν τηρεί symbol table. Επαληθεύει ότι κάθε statement ακολουθεί τη γραμματική BNF και ότι οι παρενθέσεις είναι σωστά ζευγοποιημένες.

Βασικές σχεδιαστικές επιλογές

- **Αρχικό σύμβολο `stmt_list`:** Το Bison χρησιμοποιεί τον πρώτο ορισμένο κανόνα ως αρχικό σύμβολο. Το `stmt_list` επιτρέπει κενό αρχείο (μέσω κενής παραγωγής `ε`) καθώς και οποιοδήποτε πλήθος statements.
- **Προτεραιότητα λογικών τελεστών:** Ορίζεται μέσω `%left TOK_OR / %left TOK_AND / %right TOK_NOT`, ώστε η έκφραση `a AND b OR c` να παρσάρεται ως `(a AND b) OR c`.
- **Εκτύπωση σφάλματος:** Ο `main()` διαβάζει το αρχείο γραμμή-γραμμή σε buffer, κάνει `rewind()` και ξεκινά το parsing. Η `yyperror()` εκτυπώνει τις γραμμές μέχρι το σφάλμα και τον αριθμό γραμμής.
- **Αυστηρή σειρά όρων SELECT:** Η γραμματική επιβάλλει `select → from → where → group by → order by → limit`. Διαφορετική σειρά αναγνωρίζεται ως σφάλμα.

5. Ερώτημα 2 — Σημασιολογική Ανάλυση

Το Q2 επεκτείνει τον parser με **symbol table** και σημασιολογικούς ελέγχους που υλοποιούνται αποκλειστικά σε C.

Symbol Table

Για κάθε πίνακα αποθηκεύεται το όνομα και η λίστα στηλών του (με τύπο). Η δομή χρησιμοποιεί enum ColType {TYPE_INT, TYPE_FLOAT, TYPE_VARCHAR, TYPE_UNKNOWN}.

Υλοποιούμενοι σημασιολογικοί έλεγχοι

Έλεγχος	Περιγραφή
(a) Μοναδικότητα πίνακα	Κάθε CREATE TABLE πρέπει να χρησιμοποιεί μοναδικό όνομα.
(b) Ύπαρξη πίνακα στο FROM	Ο πίνακας στο FROM πρέπει να έχει οριστεί με CREATE.
(c) Μοναδικότητα στηλών	Οι στήλες κάθε πίνακα πρέπει να είναι μοναδικές· διαφορετικοί πίνακες μπορούν να έχουν στήλη με το ίδιο όνομα.
(d) Ύπαρξη στηλών	Στήλες σε SELECT, WHERE, GROUP BY, ORDER BY πρέπει να υπάρχουν στον πίνακα FROM.
(e) Type checking	INT ↔ INT_LIT, FLOAT ↔ INT_LIT ή FLOAT_LIT (widening), VARCHAR ↔ STR_LIT. Για NOT IN: ο τύπος στήλης πρέπει να είναι συμβατός με όλα τα literals της λίστας.

Deferred checking SELECT στηλών

Ο parser βλέπει τις SELECT στήλες πριν από το FROM. Αποθηκεύονται σε buffer και ελέγχονται αφού επεξεργαστεί το FROM και οριστεί ο active πίνακας.

6. Ερώτημα 3 — JOIN, Aliases, Qualified References

Το Q3 προσθέτει υποστήριξη για **JOIN**, **AS** aliases και **qualified αναφορές** στη μορφή πίνακας.στήλη, μαζί με τους αντίστοιχους σημασιολογικούς ελέγχους.

Χαρακτηριστικό	Υλοποίηση
JOIN ... ON	Ο πίνακας JOIN πρέπει να υπάρχει στο symbol table. Τα ονόματα πινάκων/στηλών ελέγχονται.
AS alias	Alias map: πίνακας alias→table. Η sem_resolve_alias() αντιστοιχίζει alias→πραγματικό όνομα.
Qualified col_ref	Υποχρεωτικό στο ON clause (table.column). Προαιρετικό στο SELECT/WHERE/GROUP/ORDER.
Multiple JOINS	Λίστα active tables αντί για έναν. Κάθε JOIN προσθέτει τον πίνακα στη λίστα.
Ambiguity check	Bare στήλη που βρίσκεται σε >1 active tables → warning, απαιτείται qualified ref.
Reset aliases	Τα aliases επαναφέρονται στην αρχή κάθε SELECT statement.

ON clause — Υποχρεωτικά qualified references

Στο `JOIN t2 ON t1.col = t2.col` χρησιμοποιείται ξεχωριστό non-terminal `qualified_col_ref` (δέχεται μόνο `IDENT.IDENT`). Αυτό επιβάλλει ότι δεν υπάρχει αμφισημία ως προς τον πίνακα κάθε στήλης.

7. Παράδειγμα Εκτέλεσης (Q3)

```
CREATE TABLE Students (student_id INT, gpa FLOAT);

CREATE TABLE Enrollments (enrollment_id INT, student_id INT, grade VARCHAR(2));

SELECT s.student_id, e.grade

FROM Students AS s

JOIN Enrollments AS e ON s.student_id = e.student_id

WHERE s.gpa >= 3.0

AND e.grade NOT IN ('F', 'D');
```

Ακολουθία ελέγχων που εκτελείται:

- `CREATE TABLE Students` → εγγραφή στο symbol table.
- `CREATE TABLE Enrollments` → εγγραφή στο symbol table.
- `FROM Students AS s` → `sem_from_check("Students")` ✓ · εγγραφή alias `s`→`Students`.
- `JOIN Enrollments AS e ON s.student_id = e.student_id` → `sem_from_check("Enrollments")` ✓ · alias `e`→`Enrollments` · ελέγχεται `student_id` και στους δύο πίνακες.
- `SELECT deferred cols: s.student_id` ✓, `e.grade` ✓.
- `WHERE s.gpa >= 3.0` → `TYPE_FLOAT` vs `FLOAT_LIT` ✓.
- `e.grade NOT IN ('F','D')` → `TYPE_VARCHAR` vs `STR_LIT` ✓.
- Αποτέλεσμα: εκτύπωση αρχείου + «Syntactic analysis successful.»

8. Παραδείγματα Εφαρμογής

Τα screenshots επιτυχούς και ανεπιτυχούς ανάλυσης για κάθε ερώτημα (Q1, Q2, Q3) συμπεριλαμβάνονται στο συνοδευτικό **.zip** της εργασίας, στον φάκελο `screenshots/`. Καλύπτονται τουλάχιστον:

- Q1: `CREATE TABLE` ορθό · `CREATE TABLE` με λάθος (έλλειψη `,`, λάθος τύπος)
- Q1: `SELECT` ορθό · `SELECT` με λάθος σειράς όρων · `SELECT` με unmatched parentheses
- Q2: `SELECT` με υπαρκτό / ανύπαρκτο πίνακα · ανύπαρκτη στήλη · type mismatch
- Q3: `JOIN` ορθό · `JOIN` με ανύπαρκτο πίνακα · ambiguous bare column
- Q3: `AS alias` — ορθό · χρήση alias σε λάθος πίνακα

9. Σχόλια — Παραδοχές

- Τα **αρνητικά literals** (π.χ. -512, -0.5) αναγνωρίζονται απευθείας από τον lexer ως INT_LIT / FLOAT_LIT με αρνητικό πρόσημο.
- Το **LIMIT** δέχεται μόνο αυστηρά θετικό INT_LIT· μηδέν και αρνητικοί ακέραιοι αναγνωρίζονται ως σφάλμα στο semantic check.
- Τα **block comments** επιτρέπονται μεταξύ tokens, αλλά όχι εντός ενός token (π.χ. Se/*...*/lect είναι σφάλμα).
- Στο Q2 το col_ref δέχεται **μόνο bare IDENT** — το table.column επανέρχεται στο Q3.
- Στο Q3, αν χρησιμοποιείται alias για κάποιον πίνακα, **όλες** οι αναφορές στις στήλες του πρέπει να γίνονται μέσω του alias (alias.column).
- Οι σημασιολογικοί έλεγχοι Q2 & Q3 υλοποιούνται αποκλειστικά σε **C** (χωρίς χρήση C++ containers).
- Στη **FLOAT / INT widening**: η σύγκριση FLOAT_col >= 42 (INT literal) επιτρέπεται λόγω implicit type widening. Το αντίστροφο (INT_col >= 3.14) δεν επιτρέπεται.